Xi'an Jiaotong-Liverpool University
西交利物浦大學

SQL - Tables and Data Part 1

Jianjun Chen

Relational Database Management System

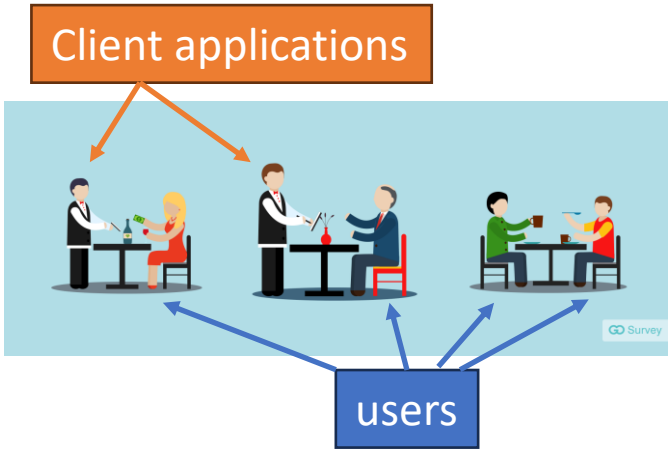
- Relational database management systems (RDBMS) are based on the relational model.
  - Many relational operations are supported.

- Oracle
- DB2
- MySQL and MariaDB
- Ingres
- PostgreSQL
- Microsoft SQL Server
- MS Access

RDBMS and Related Software

- DBMS is a <u>server application</u> that usually runs on a server.
  - MySQL, MariaDB, PostgreSQL, Oracle…
  - No fancy user interface.
- <u>Client applications</u> communicate with this server application.
  - Clients designed for managing DBMS: DBeaver, MySQL workbench, PhpMyAdmin
  - Clients that uses DBMS for services: Your favourite game, Hotel booking website, software on ATM machines.
  - Interfaces of clients varies.

server



Match the following items

MariaDB	Database server
DBeaver	
PostgreSQL	Client application of DB
Smart watch app	
MySQL workbench	Human user of DB
Microsoft SQL Server	
Someone using ATM	DB designer
PhpMyAdmin	
Someone playing an online game	Client application developer
Oracle DB	
Hotel booking app developer	None of the above matches
Hotel booking database developer	

Database Language

- Database has its own “programming language” called Structured Query Language (SQL)
- SQL has three important parts:
  - **Data Definition Language** (DDL): Allows users to specify the data types and structures and the constraints on the data to be stored in the database.
  - **Data Manipulation Language** (DML): Allows users to insert, update, delete, and retrieve data from the database
  - **Data Control Language** (DCL): Deals with the rights, permissions, and other controls of the database system.
- Flow control (if, else, loops) is included in extensions like PL/SQL and Transact-SQL.

Contents

- Today, we will cover the DDL and a small portion of DML:

- Creating Tables
  - Syntax
  - Data types.
  - Column options

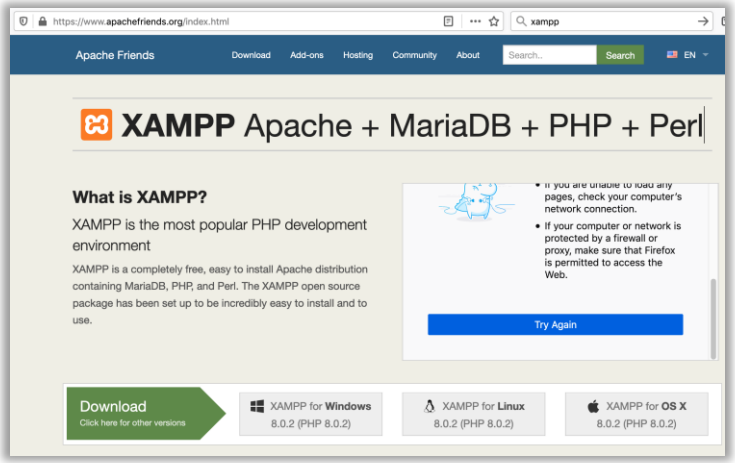
- Tuple Manipulation
  - Insert, update, delete

XAMPP

- MySQL and PhpMyAdmin are used in our labs.
  - It is recommended to install XAMPP on your own computer if you want the same environment.

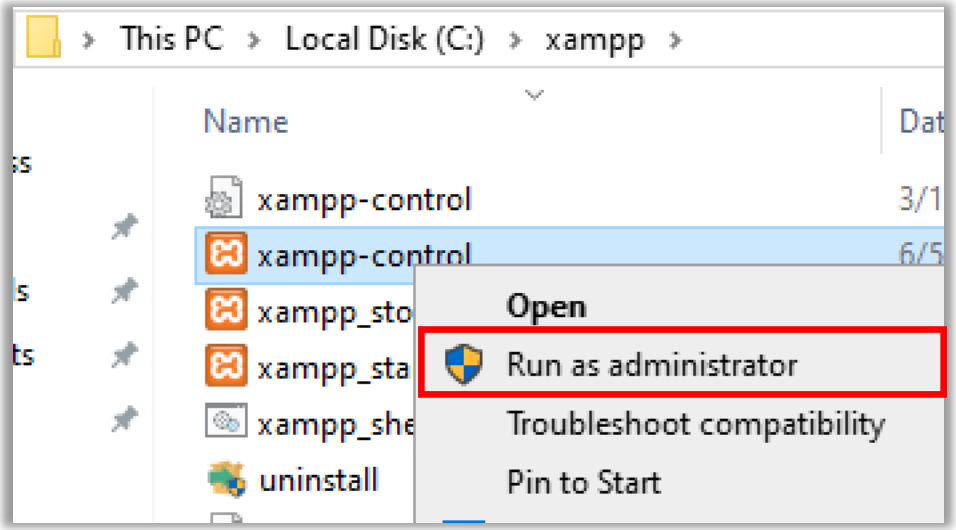
- XAMPP is a collection of software, including MySQL and PhpMyAdmin.

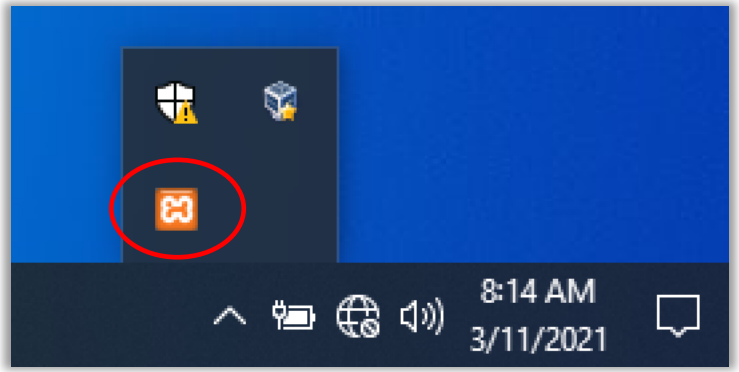
- The installer program will be shared on LearningMall.
  - You may also download from here.
  - Connection can be slow though



Setting up XAMPP (Windows)

- Install XAMPP -> Run “xampp-control.exe”
  - Always run “xampp-control.exe” as administrator.

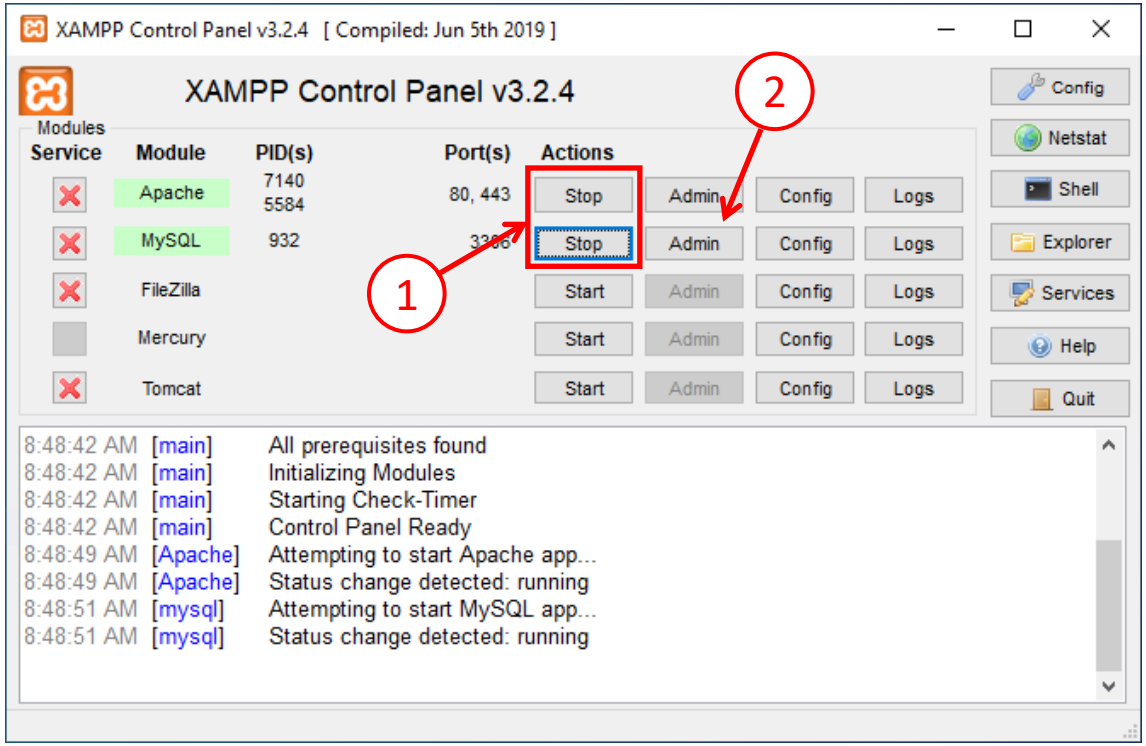




- You can start multiple instances of xampp-control, but don’t do that!
  - Check the system tray before starting xampp. If there’s one running, just double click it to show its main UI.

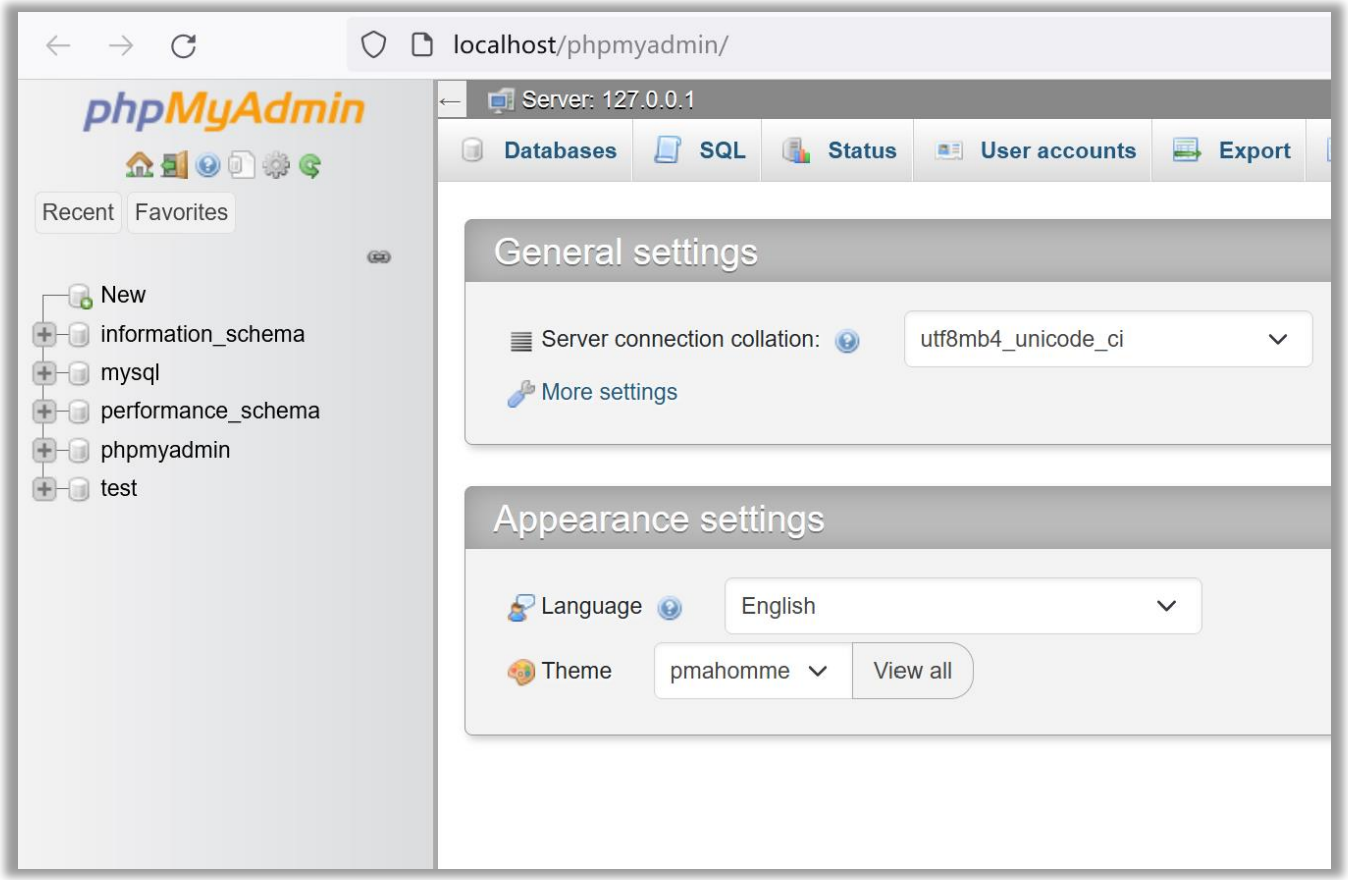
Starting Database (Windows)

- Step 1: start both Apache and MySQL
  - Apache in XAMPP provides PhpMyAdmin.
- Step 2: click the ‘Admin’ button of MySQL.



PhpMyAdmin

- If you see the website below, then MySQL is ready.

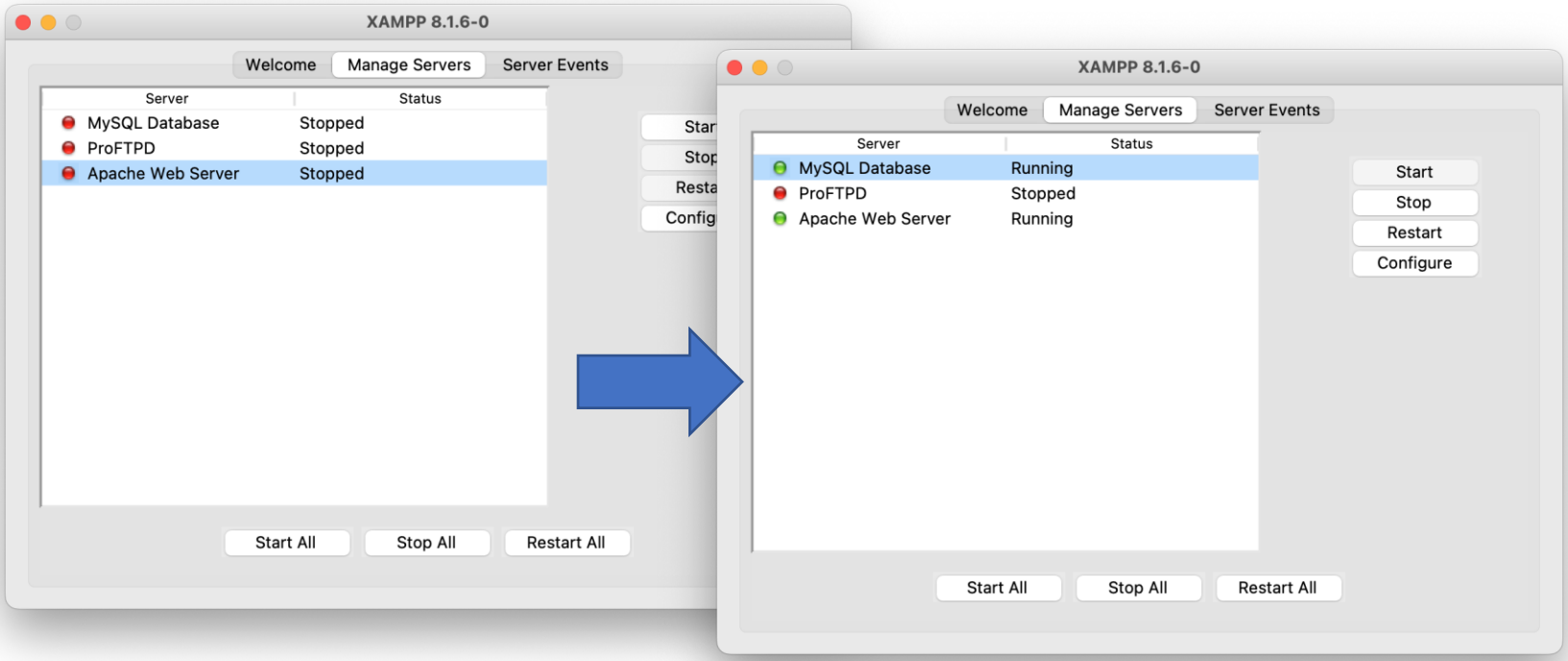


Setting up XAMPP (Mac)

- Install XAMPP -> launch ‘manager-osx’ in Applications.

Setting up XAMPP (Mac)

- In ‘Manage Servers’, start MySQL and Apache.
- Wait until MySQL and Apache has ‘running’ status.

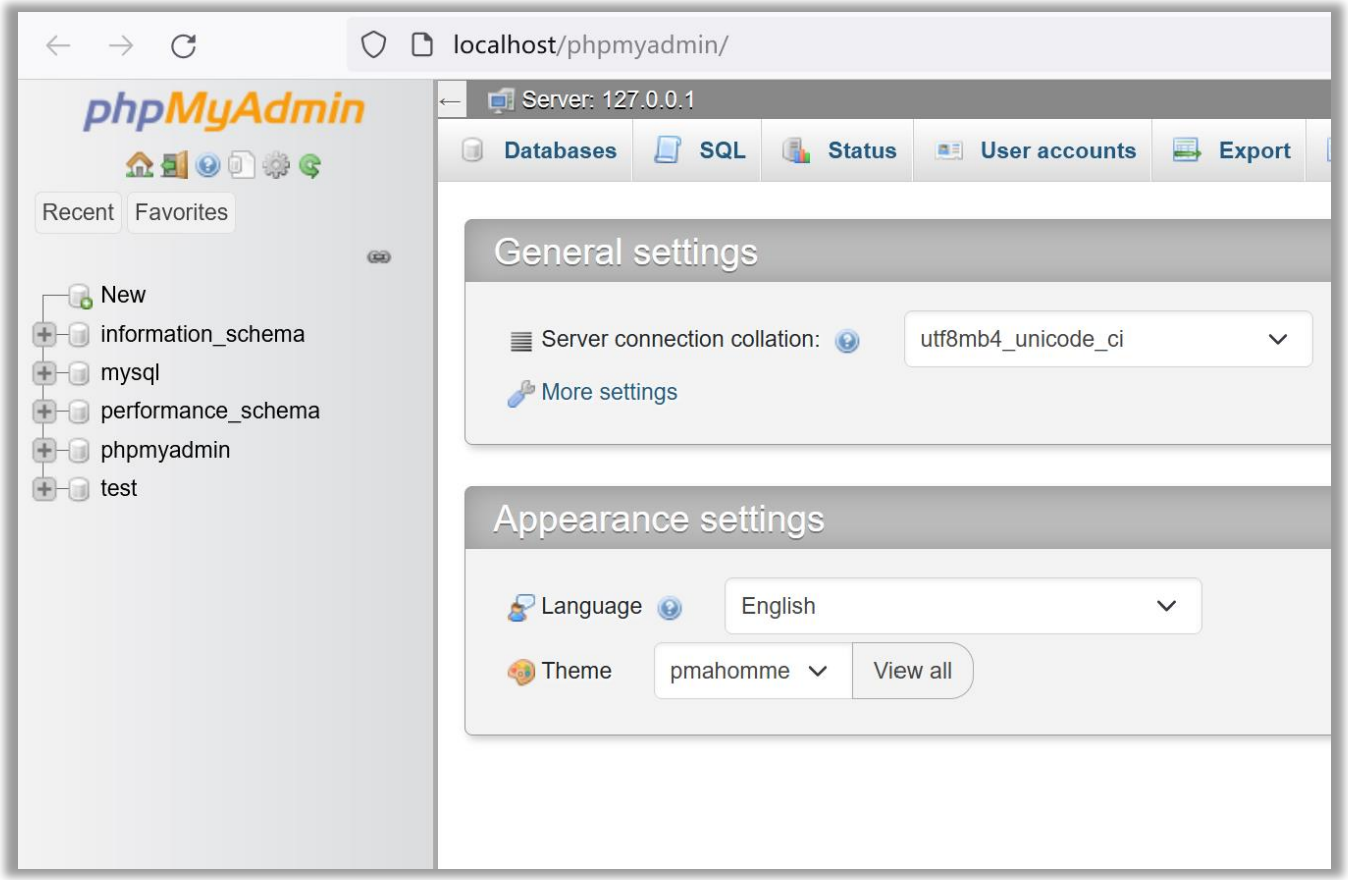


Setting up XAMPP (Mac)

- Go back to ‘Welcome’ and click ‘Go to Application’.
  - Your web browser should appear with an interface like below.
- Click phpMyAdmin on top right corner

PhpMyAdmin

- If you see the website below, then MySQL is ready.



Having Problems?

- One may encounter problems when setting up XAMPP.
- Please DO NOT ask them via emails.
  - Many issues should be solved face-to-face.
- Asking during office hours or lab sessions.

Playing with SQL

Create schemas

SQL Format in the Slides

- SQL statements will be written in
  - **BOLD COURIER FONT**

- SQL keywords are case-insensitive, but my slides will present them in uppercase for clarity.
- Table names, column names etc. can be configured to be case-sensitive. For example:

```
SELECT (sName) FROM Student;
```

- SQL statements should end with semi-colons.

- Two dashes followed by a space will comment out that line.

```
-- commented out
--not correctly commented out
```

Creating a Schema

- First, we need to create a schema
  - **CREATE SCHEMA** `name;`
  - **CREATE DATABASE** `name;`

  Same

- A schema consists of tables (relations), views, domains, assertions, collations, translations, and character sets.
  - All have same owners.

- You can create a schema accessible only to IT department and another schema accessible only to marketing department.
  - Staff cannot see the contents of the schema of another department
  - How to do it is not covered in this module.

phpMyAdmin

Recent Favorites

- New
- cse103
- information_schema
- mysql
- performance_schema
- phpmyadmin

Server: 127.0.0.1

Databases | SQL | Status | User accounts | Export | More

Run SQL query/queries on server "127.0.0.1":

```
1 CREATE SCHEMA mySchema;
```

Clear | Format | Get auto-saved query

☐ Bind parameters

Bookmark this SQL query:

[Delimiter ;] ☑ Show this query here again ☐ Retain query box ☐ Rollback when finished

☑ Enable foreign key checks

Go

phpMyAdmin

Recent Favorites

- New
- cse103
- information_schema
- myschema
- mysql
- performance_schema
- phpmyadmin

Server: 127.0.0.1

Databases | SQL | Status | User accounts | Export | More

Show query box

✔ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0019 seconds.)

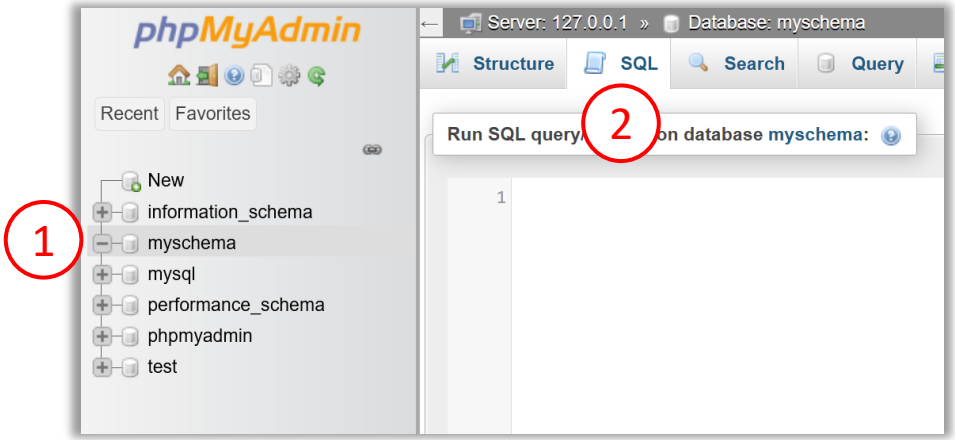
```
CREATE SCHEMA mySchema
```

[Edit inline] [Edit] [Create PHP code]

⚠ Error: #1046 No database selected

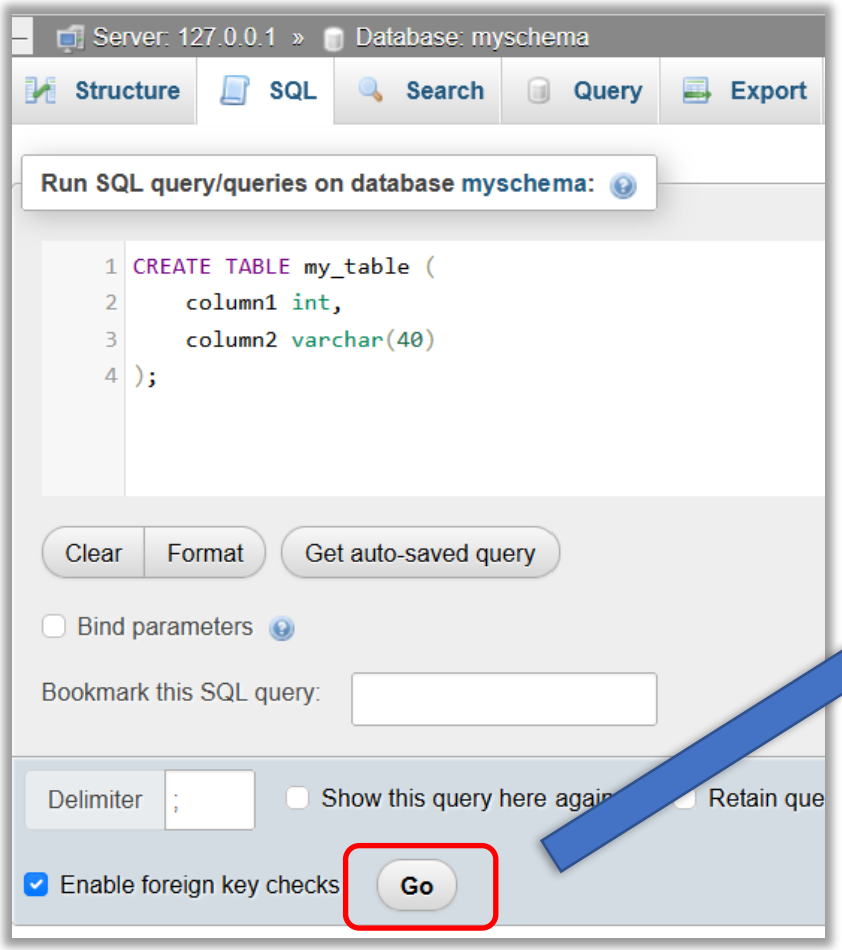
Creating the First Table

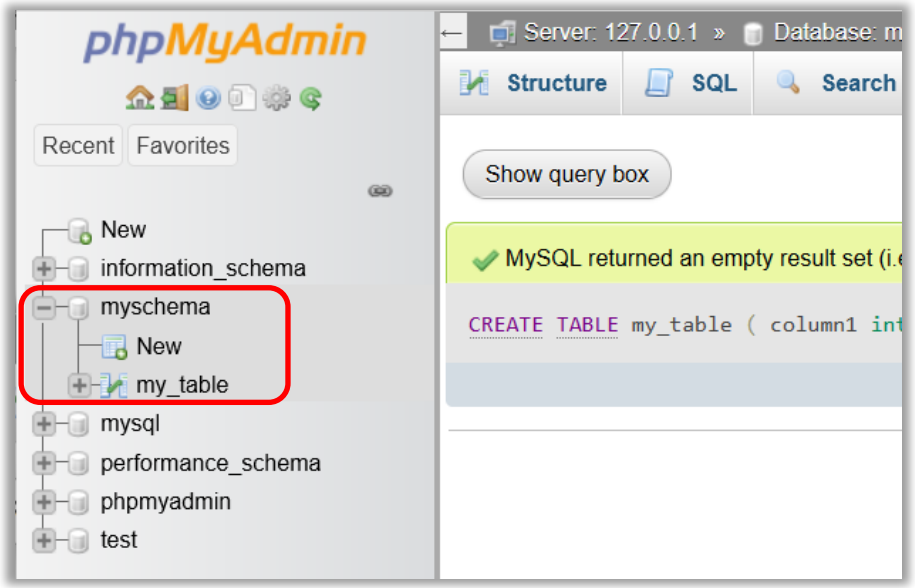
- Click “myschema”, then click SQL.



- Enter the following code:

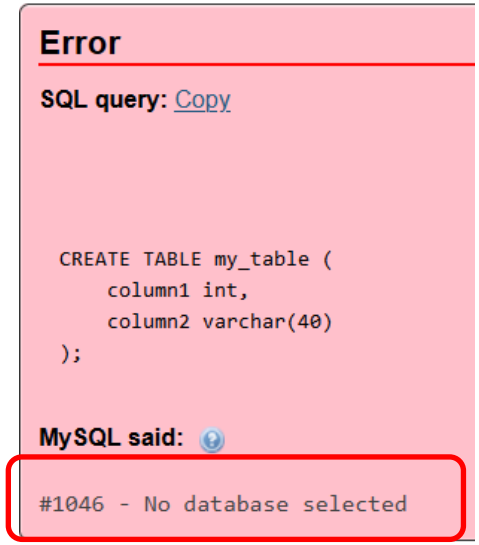
```
CREATE TABLE my_table (
    column1 int,
    column2 varchar(40)
);
```

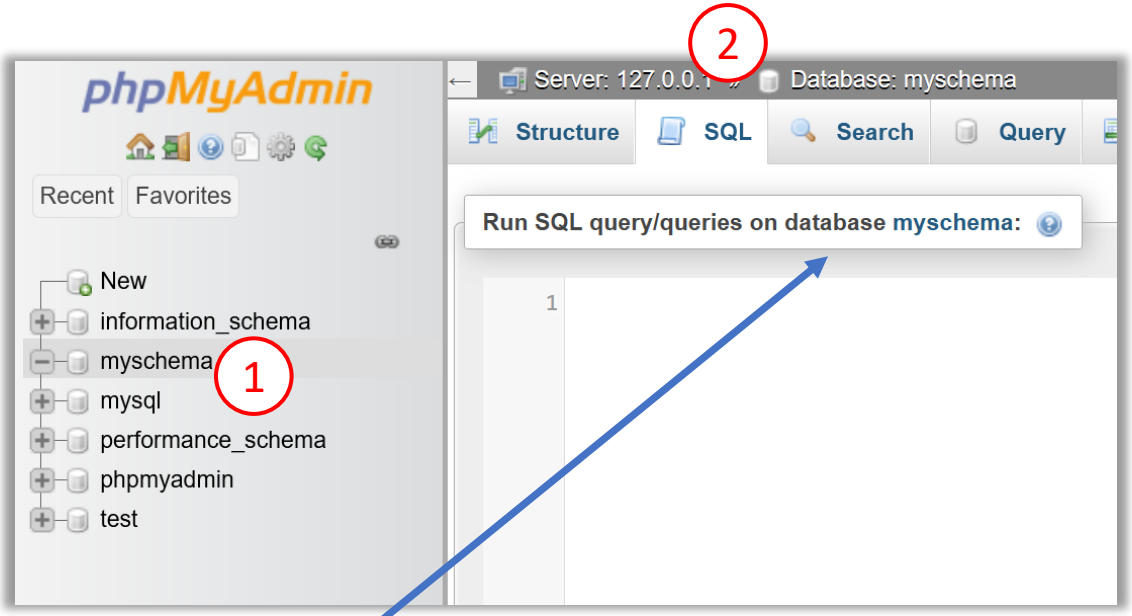

Server: 127.0.0.1 » Database: myschema
Structure
SQL
Search
Query
Export
Run SQL query/queries on database myschema:
CREATE TABLE my_table (
column1 int,
column2 varchar(40)
);
Clear
Format
Get auto-saved query
Bind parameters
Bookmark this SQL query:
Delimiter
;
Show this query here again
Retain que
Enable foreign key checks
Go


phpMyAdmin
Recent
Favorites
New
information_schema
myschema
New
my_table
mysql
performance_schema
phpmyadmin
test
Server: 127.0.0.1 » Database: m
Structure
SQL
Search
Show query box
MySQL returned an empty result set (i.e
CREATE TABLE my_table (column1 int

A Common Error

- If you see the “No database selected” error, remember to click on the schema first, then click the SQL tab.





- Current schema is indicated here.
- A table cannot be created outside of a schema.

Keyword USE

- Alternatively, you can use the keyword `USE`.
- Syntax:

```
USE schema_name;
```

- It forces all subsequent SQL commands to be executed on the schema you specified.

Run SQL query/queries on server "127.0.0.1":

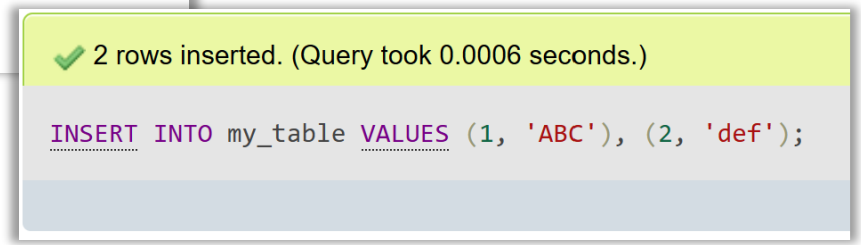
```
use myschema;
CREATE TABLE another_table (
    column1 int,
    column2 varchar(40)
);
```

Clear | Format | Get auto-saved query

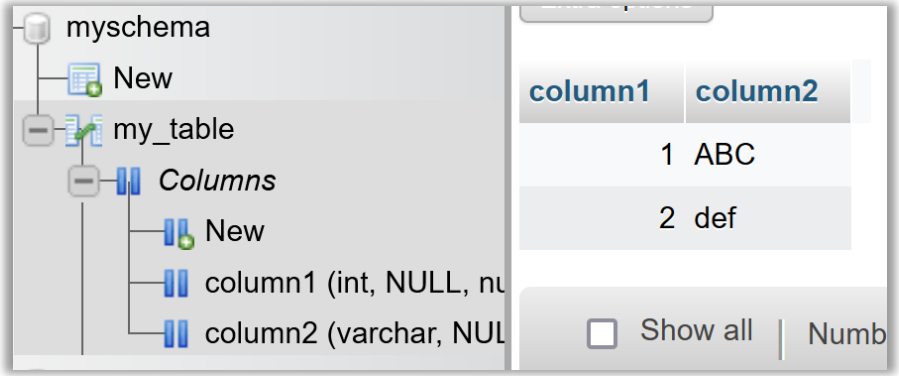
Inserting Values

- Enter the following code to insert two rows into the table we just created:

```
INSERT INTO my_table VALUES
(1, 'ABC'), (2, 'def');
```



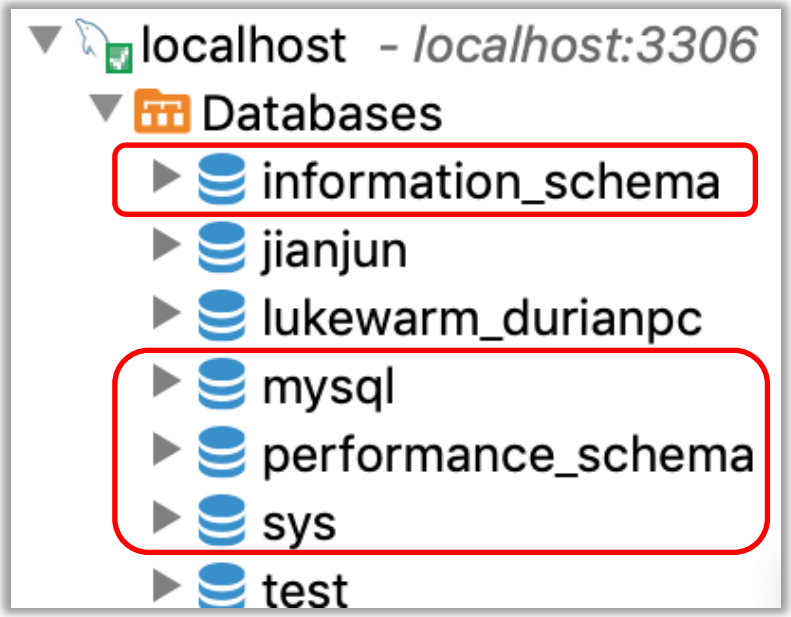
- Click on my_table to see its new content.



Some Notes

- Avoid creating or modifying tables in the following schemas:
  - information_schema
  - mysql
  - performance_schema
  - sys

- They are automatically created for database administration purpose.



Some Notes

- Error messages in MySQL are not always informative.

Run SQL query/queries on database test:

```
1 create table `sometable`
2 (
3     column1 INT NOT NULL,
4     column2 TEXT,
5 );
```

Error

Static analysis:

1 errors were found during analysis.

1. A symbol name was expected! (near ")" at position 72)

SQL query: Copy

create table `sometable` (column1 INT NOT NULL, column2 TEXT,)

MySQL said:

```
#1064 - You have an error in your SQL syntax; check t
at line 5
```

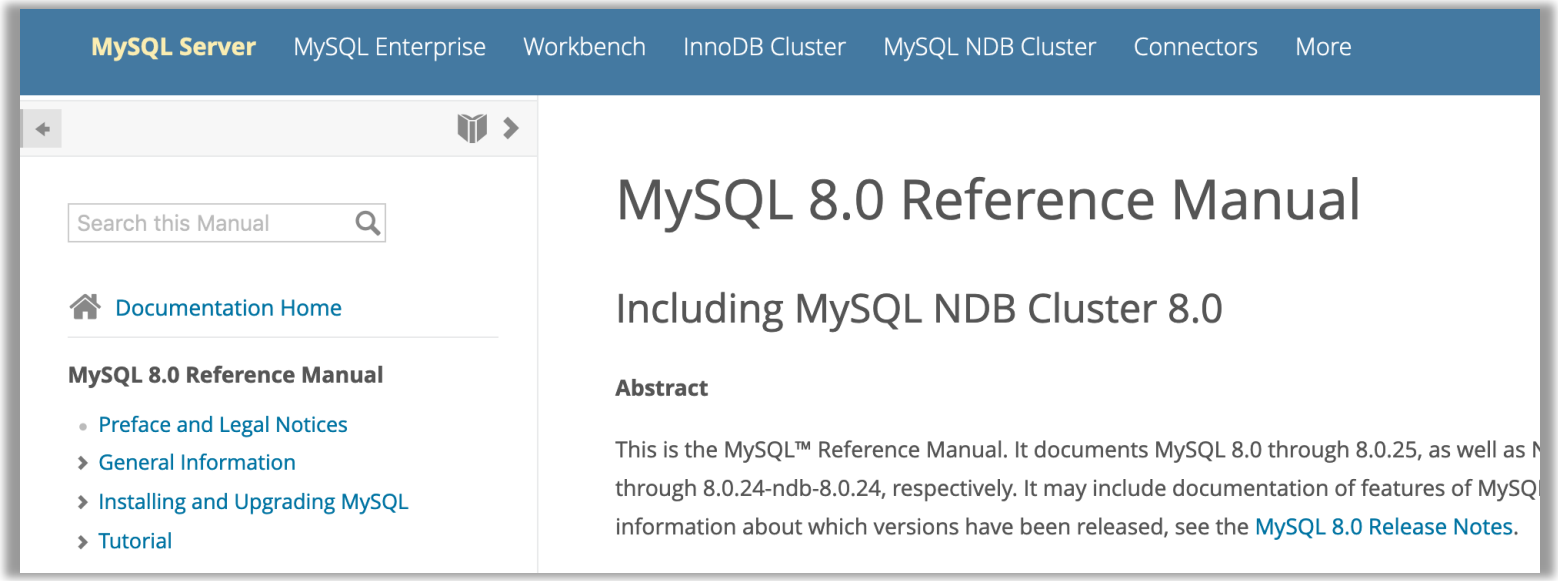
- As a result, remembering the correct syntax is very important when writing SQL statements.
- You may check the official manual for detailed syntax.

The Official Manual

- Being able to check the official manual is a very important skill.

https://dev.mysql.com/doc/refman/8.0/en/

- Looking up “how to xxxx” on Google/Baidu does not always work.



SQL: Creating Tables

Syntax of "`CREATE TABLE`"

Data types in SQL

Syntax

To create tables in SQL, you need the “Create table” statement, the general syntax is shown below:

语法

```
CREATE TABLE [IF NOT EXISTS] name (
    col-name datatype [col-options],
    :
    col-name datatype [col-options],
    [constraint-1],
    :
    [constraint-n]
);
```

Contents between [] are optional

Syntax of INSERT

- The syntax below inserts values into a table:

```
INSERT INTO tablename VALUES (val1, val2, …);
```

- The order of "val1, val2, …" must match the order of columns of the table.
- For example:

```
CREATE TABLE my_table (
    column1 int,
    column2 varchar(40)
);
```

```
INSERT INTO my_table VALUES
(1, 'ABC'), (2, 'def');
```

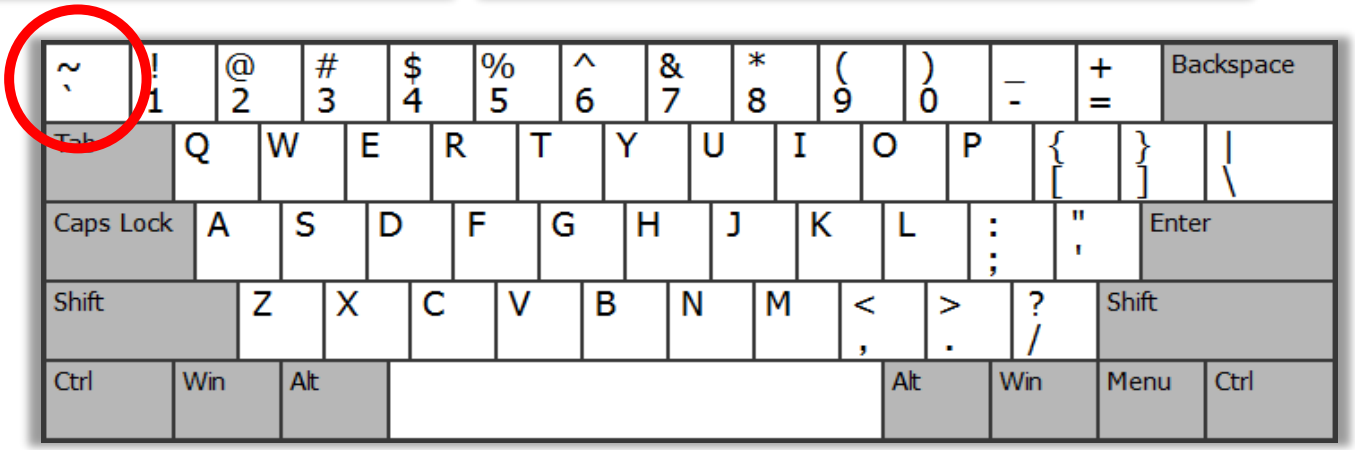
Tuple 1 Tuple 2

Table and Column Names

- In SQL, table or column names can have spaces, but it is strongly not recommended.
- If you insist to do so, you must enclose the name with a pair of backtick (`) symbols.

```
CREATE TABLE mytable (
    columnA INT,
    columnB CHAR(11)
);
```

```
CREATE TABLE `my table` (
    `column A` INT,
    `column B` CHAR(11)
);
```



Numerical Data Types (Integers)

- **SMALLINT**
  - Use 2 Bytes of memory
  - range: -32768 to +32767

Forgot about Bits, Bytes, KB, … ?
https://web.stanford.edu/class/cs101/bits-bytes.html

- **INT** or **INTEGER**
  - Use 4 Bytes of memory
  - range: -2147483648 to +2147483647
  - A typical choice for integer.

- **BIGINT**
  - Use 8 Bytes of memory.

```
CREATE TABLE `staff` (
    `name` VARCHAR(12),
    `staff_id` INT
);
```

Numerical Data Types (Fixed Point)

- **`DECIMAL[(M[,D])]`** or **`NUMERIC[(M[,D])]`**
  - Fixed point number.

Things between [] are optional

- Example:

```
CREATE TABLE `staff` (
    `name` VARCHAR(12),
    `staff_id` INT(11),
    `salary` DECIMAL(5,2)
);
```

Decimal(size,d)

D

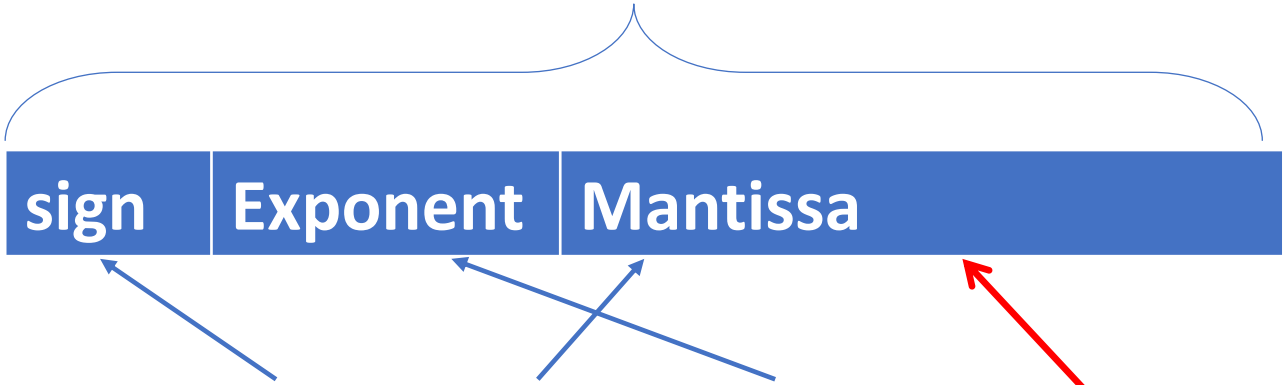
12345666666.7777

M

Numerical Data Types (Float)

- **`FLOAT(p)`**
  - Floating point number in *IEEE 754 standard*
  - prepresents the precision in bits. (Mantissa size)
  - MySQL will automatically choose single precision or double precision based on the value of p.

Total number of bits is 32 for float and 64 for double



$$\pm (1.m) \times 2^{E}$$

This is what p refers to

String Types

- **CHAR [(M)]**
  - A fixed-length (**M** is the length, 0 ~ 255) string
  - **CHAR** alone without M is the same as **CHAR(1)**.
  - Always right-padded with spaces to the specified length when stored.
  - E.g. if you store **'A'** to **CHAR(5)**, it will actually be **'A '** inside the memory. But when you retrieve the value, the trailing spaces will be removed automatically (this behaviour can be turned on or off).

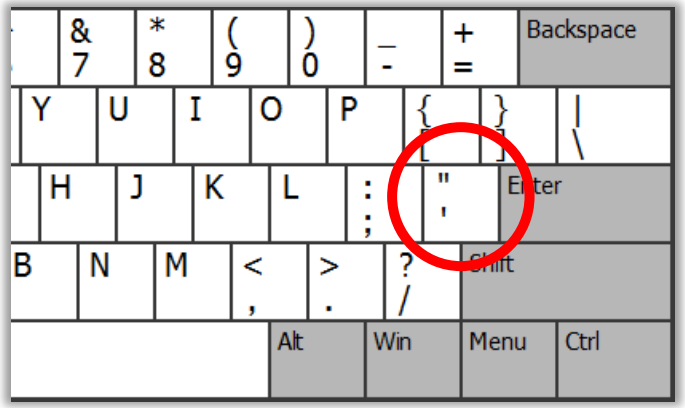
- **VARCHAR(M)**
  - A variable-length string.
  - The range of M is 0 to 65,535.

```
CREATE TABLE `staff` (
    `name` VARCHAR(60),
    `id_card` CHAR(6)
);
```

- For MySQL:
  - **CHAR** is faster, but occupies more memory
  - **VARCHAR** is slower, but occupies less memory

String Values

- String values in SQL are surrounded by single quotes:
  - **`'I AM A STRING'`**
- Single quotes within a string are doubled or escaped using \
  - **`'I''M A STRING'`**
  - **`'I\'M A STRING'`**
  - **`''`** - is an empty string
- In MySQL, double quotes also work
  - A non-standard feature.
- Example (after creating the table)




```
CREATE TABLE `staff` (
    `name` VARCHAR(60),
    `id_card` CHAR(6)
);
insert into `staff` values ('Daryl', 'STF001');
```

Question

- Which of the following strings inserted contains the single quote character? Which of them is/are incorrectly formatted?

```
INSERT INTO staff VALUES
        ('Daryl', '00A001'),
        ('Mary O'Brien', '00B002'),
        ('Kate La\'Keisha', '00C003'),
        ('Perry O''Connell', '00D004'),
        ('Daryl\''s friend', '00E005');
```

Question 2

- All columns with numeric types (INTEGER, DECIMAL, FLOAT etc.) can be defined with VARCHAR or CHAR instead.

```
CREATE TABLE student (
        id INT,
        name VARCHAR(60),
        age INT,
        tel INT
);
```

```
CREATE TABLE student (
        id VARCHAR(11),
        name VARCHAR(60),
        age VARCHAR(3),
        tel VARCHAR(18)
);
```

- Any disadvantage(s) you can think of?
- Any advantage(s) in some cases?

Date and Time

- **DATE**
  - The supported range is '1000-01-01' to '9999-12-31'.
  - MySQL displays DATE values in 'YYYY-MM-DD' format.

- **DATETIME**[(fsp)]

The number of digits is defined by fsp

  - The supported range is '1000-01-01 00:00:00.000000' to '9999-12-31 23:59:59.999999'
  - Display format in MySQL: 'YYYY-MM-DD hh:mm:ss[.fraction]'

- Example:

```
CREATE TABLE `staff` (
    `name` VARCHAR(60),
    `id_card` CHAR(6)
    `recruit_date` DATE,
    `last_login` DATETIME
);
```

Date and Time

- **TIMESTAMP**
  - Similar to DATETIME but stores the time as UTC time.
  - When date time is looked up, the stored time will be converted to the time of the current timezone of the client.
  - How to illustrate their difference? Enter the following table with data:

```sql
create table `ts_test` (
    `time1` timestamp,
    `time2` datetime
);


insert into `ts_test` values
('2020-01-08 08:00:00', '2020-01-08 08:00:00');
```

Date and Time

- Assume your current timezone is UTC+8 (Beijing).
  - Look up the data in ts_test, you will find:

	time1	time2
1	2020-01-08 08:00:00	2020-01-08 08:00:00

- Now, in the system, change your timezone to UTC+0 (London)
  - Look up the data in ts_test again, you will now find:

	time1	time2
1	2020-01-08 00:00:00	2020-01-08 08:00:00

Missing Values

- Sometimes a tuple does not have a valid value for a column.
  - Jason below does not use telephone at all.

Staff

ID	Name	tel	mobile
1	Jason		1371111111

- Sometimes we don't know the value of a tuple.
  - Wendy's age is unknown.

Staff

ID	Name	age	mobile
1	Wendy		1381111111

NULL and Special Values

- You may use NULL or special values to indicate the situation.
- Special value example:
  - No one has an age of 0.

Staff

ID	Name	age	mobile
1	Wendy	0	1371111111

占位符

- NULL is a special placeholder that can be used in columns (that allow NULL) with different data types.
  - It has special behaviors when arithmetic operations are involved.
  - Covered in detail in a later lecture.

Staff

ID	Name	age	mobile
1	Wendy	NULL	1371111111

Column Options

```
col-name datatype [col-options]
```

- **`NOT NULL`**:
  - values of this column cannot be null.
- **`UNIQUE`**:
  - each value must be unique
- **`DEFAULT`** `value`:
  - Default value for this column if not specified by the user.
  - Does not work in MS Access.

Example: age INT DEFAULT 12

Column Options

- **AUTO_INCREMENT**

```
CREATE TABLE Persons (
    Id INT AUTO_INCREMENT,
    ...
)
```

- Must be applied to a key column (primary key, unique key)
- a value, usually **max(col) + 1**, is automatically inserted when data is added.
  - First value is 1.
  - `INSERT INTO Persons VALUES (null, …);`
- You can also manually provide values to override this behaviour.
  - E.g., add the following line below the CREATE TABLE statement:

```
ALTER TABLE Persons AUTO_INCREMENT = 100;
```

Example

```
CREATE TABLE Persons (
    id INT UNIQUE NOT NULL AUTO_INCREMENT,
    lastname VARCHAR(255) NOT NULL,
    firstname VARCHAR(255),
    age INT DEFAULT 12,
    city VARCHAR(255)
) AUTO_INCREMENT = 5;
```

Extended Reading - Implicit Default Values

- When a `DEFAULT` option is not used, MySQL may give implicit default values depending on the column data type:
  - If the column can take `NULL` as a value, the column is defined with an explicit `DEFAULT NULL` clause.
  - If the column cannot take `NULL` as a value, MySQL defines the column with no explicit `DEFAULT` clause.

```
CREATE TABLE t (
    i INT NOT NULL
);
```

```
INSERT INTO t VALUES();
INSERT INTO t VALUES(DEFAULT);
INSERT INTO t VALUES(DEFAULT(i));
```

All statements above gives error in strict mode

- Please check the documentation for details: https://dev.mysql.com/doc/refman/8.0/en/data-type-defaults.html.

Tuple Manipulation

INSERT, UPDATE, DELETE

More Syntax of INSERT

- INSERT add rows into the database:

```
INSERT INTO tablename (col1, col2, …)
    VALUES (val1, val2, …),
           :
           (val1,val2,val3);
```

- The order of column names must be consistent with the order of values.

- If you are adding a value to every column, you don't have to list them:

```
INSERT INTO tablename VALUES (val1, val2, …);
```

INSERT: Example

If we try to run the following statements in MySQL, what will the data in the table look like?

```
INSERT INTO `Employee`
    (`ID`, `Name`, `Salary`)
VALUES (2, 'Mary', 26000);
```

```
INSERT INTO Employee
    (Name, ID)
VALUES ('Mary', 2);
```

```
INSERT INTO Employee
VALUES (2, 'Mary', 26000),
       (3, 'Max', 19000);
```

Employee

ID	Name	Salary

INSERT: Example

If we try to run the following statements in MySQL, what will the data in the table look like?

```
INSERT INTO `Employee`
    (`ID`, `Name`, `Salary`)
VALUES (2, 'Mary', 26000);
```

```
INSERT INTO Employee
    (Name, ID)
VALUES ('Mary', 2);
```

```
INSERT INTO Employee
VALUES (2, 'Mary', 26000),
       (3, 'Max', 19000);
```

Employee

ID	Name	Salary
2	Mary	26000
2	Mary	
2	Mary	26000
3	Max	19000

UPDATE

- Changes values in specified rows based on WHERE conditions

```
UPDATE table-name
   SET col1 = val1 [,col2 = val2…]
   [WHERE condition]
```

- All rows where the condition is true have the columns set to the given values.
- If no condition is given <u>all rows</u> are changed.
- Values are constants or can be computed from columns.
- Details of the WHERE clause will be taught later.

UPDATE: Example

Employee

ID	Name	Salary
1	John	25000
2	Mary	26000
3	Mark	18000
4	Anne	22000

```
UPDATE Employee
SET
    Salary = 15000,
    Name = 'Jane'
WHERE ID = 4;
```

Employee

ID	Name	Salary
1	John	25000
2	Mary	26000
3	Mark	18000
4	Jane	15000

```
UPDATE Employee
SET Salary =
    Salary * 1.05;
```

Employee

ID	Name	Salary
1	John	26250
2	Mary	27300
3	Mark	18900
4	Anne	23100

Question

- One year has passed, the database maintainer needs to increase the age of all employees.
- However, some age values are special values indicating that we don't know the ages of these people.

Staff

ID	Name	age	mobile
1	Wendy	0	1371111111
2	John	23	1382222222
3	Steven	31	1773333333
4	Larry	24	1334444444
…			

- Your solution to this?

DELETE

- Removes all rows, or those which satisfy a condition

```
DELETE FROM
     table-name
     [WHERE condition]
```

- If no condition is given then ALL rows are deleted.

DELETE: Example

Employee

ID	Name	Salary
1	John	25000
2	Mary	26000
3	Mark	18000
4	Anne	22000

```
DELETE FROM
Employee WHERE
Salary > 20000;
```

Employee

ID	Name	Salary
3	Mark	18000

```
DELETE FROM
Employee;
```

Employee

ID	Name	Salary

Final Example

- Convert the following relation into SQL query.
- Choose appropriate data types for columns.

Branch

branchNo	street	city	postCode
B005	22 Deer Rd	London	SW1 4EH
B007	16 Argyll St	Aberdeen	AB2 3SU
B003	163 Main St	Glasgow	G11 9QX
B004	32 Manse Rd	Bristol	BS99 1NZ
B002	56 Clover Dr	London	NW10 6EU

- Then update all “B005”s to “B009”
- Delete all tuples that has “B003”